

AI-Powered Postal Delivery Performance Analysis using Machine Learning, RAG and LLM

1. Abstract

This project analyzes more than 163,000 postal delivery records to understand delivery efficiency, identify bottlenecks, and predict delivery performance. The original machine learning pipeline was enhanced with Retrieval-Augmented Generation (RAG) and a Large Language Model (LLM) to create an intelligent conversational analytics assistant.

This project analyzes more than 163,000 postal delivery records to understand delivery efficiency, identify bottlenecks, and predict delivery performance. The original machine learning pipeline was enhanced with Retrieval-Augmented Generation (RAG) and a Large Language Model (LLM) to create an intelligent conversational analytics assistant.

2. Problem Statement

Postal organizations handle millions of deliveries every day. Delays and inefficiencies impact customer satisfaction and operational costs. The objective of this project is to predict delivery performance and enable natural language interaction with delivery data.

Postal organizations handle millions of deliveries every day. Delays and inefficiencies impact customer satisfaction and operational costs. The objective of this project is to predict delivery performance and enable natural language interaction with delivery data.

3. Objectives

1. Perform exploratory data analysis. 2. Engineer useful features. 3. Build predictive models. 4. Compare model performance. 5. Implement semantic search using embeddings. 6. Build a RAG pipeline. 7. Integrate an LLM for natural language responses.

1. Perform exploratory data analysis. 2. Engineer useful features. 3. Build predictive models. 4. Compare model performance. 5. Implement semantic search using embeddings. 6. Build a RAG pipeline. 7. Integrate an LLM for natural language responses.

4. Dataset Description

The dataset contains more than 163,000 records and includes information such as state, district, office type, and delivery metrics. Missing values, duplicates, and inconsistent values were cleaned during preprocessing.

The dataset contains more than 163,000 records and includes information such as state, district, office type, and delivery metrics. Missing values, duplicates, and inconsistent values were cleaned during preprocessing.

5. Data Cleaning and Preprocessing

Performed null value handling, duplicate removal, datatype correction, outlier detection, and feature transformations using Pandas and NumPy.

Performed null value handling, duplicate removal, datatype correction, outlier detection, and feature transformations using Pandas and NumPy.

6. Exploratory Data Analysis

EDA was used to discover delivery patterns, state-wise performance, district-wise trends, office-type distribution, and potential bottlenecks.

EDA was used to discover delivery patterns, state-wise performance, district-wise trends, office-type distribution, and potential bottlenecks.

7. Feature Engineering

Created derived features and transformed categorical variables into machine learning compatible formats to improve predictive performance.

Created derived features and transformed categorical variables into machine learning compatible formats to improve predictive performance.

8. Machine Learning Models

Random Forest, XGBoost, and CatBoost were trained and evaluated. Hyperparameter tuning was performed to improve prediction quality.

Random Forest, XGBoost, and CatBoost were trained and evaluated. Hyperparameter tuning was performed to improve prediction quality.

9. Model Performance

The best model achieved an R^2 score of 0.92, indicating strong predictive capability for delivery performance estimation.

The best model achieved an R^2 score of 0.92, indicating strong predictive capability for delivery performance estimation.

10. Why RAG Was Added

Traditional ML models can predict values but cannot answer questions in natural language. RAG was introduced to allow users to query the dataset conversationally.

Traditional ML models can predict values but cannot answer questions in natural language. RAG was introduced to allow users to query the dataset conversationally.

11. Document Creation

Each postal record was converted into textual documents containing district, state, office type, and delivery rate information.

Each postal record was converted into textual documents containing district, state, office type, and delivery rate information.

12. Embeddings

Sentence Transformers were used to convert text documents into numerical vectors capturing semantic meaning.

Sentence Transformers were used to convert text documents into numerical vectors capturing semantic meaning.

13. Vector Database

FAISS was used to store embeddings and perform high-speed similarity search across thousands of records.

FAISS was used to store embeddings and perform high-speed similarity search across thousands of records.

14. Retrieval Stage

User questions are converted into embeddings and matched against stored vectors to retrieve relevant records.

User questions are converted into embeddings and matched against stored vectors to retrieve relevant records.

15. LLM Integration

Qwen2-0.5B-Instruct was integrated to convert retrieved records into natural language answers.

Qwen2-0.5B-Instruct was integrated to convert retrieved records into natural language answers.

16. RAG Workflow

User Question → Embedding Generation → FAISS Retrieval → Context Construction → LLM Response Generation.

User Question → Embedding Generation → FAISS Retrieval → Context Construction → LLM Response Generation.

17. Example Query

Question: Which districts have excellent delivery performance? Answer: The district with excellent delivery performance is Alluri Sitharama Raju.

Question: Which districts have excellent delivery performance? Answer: The district with excellent delivery performance is Alluri Sitharama Raju.

18. Technologies Used

Python, Pandas, NumPy, Scikit-learn, XGBoost, CatBoost, Sentence Transformers, FAISS, Hugging Face Transformers, PyTorch, Jupyter Notebook.

Python, Pandas, NumPy, Scikit-learn, XGBoost, CatBoost, Sentence Transformers, FAISS, Hugging Face Transformers, PyTorch, Jupyter Notebook.

19. Applications

This system can be extended to logistics, courier analytics, supply chain optimization, smart customer support, and government postal operations.

This system can be extended to logistics, courier analytics, supply chain optimization, smart customer support, and government postal operations.

20. Future Enhancements

Future improvements include web deployment, dashboard integration, larger LLMs, GPU acceleration, and real-time data ingestion.

Future improvements include web deployment, dashboard integration, larger LLMs, GPU acceleration, and real-time data ingestion.

21. Conclusion

The project successfully combines traditional machine learning with modern Generative AI techniques to create an intelligent postal analytics assistant.

The project successfully combines traditional machine learning with modern Generative AI techniques to create an intelligent postal analytics assistant.